

Vemi Portal

Vemi Portal is a comprehensive React-based admin and management dashboard designed for managing draws, prizes, users, and engagement reports.

□ Getting Started

Prerequisites

- **Node.js:** Ensure you have a modern version of Node.js installed.
- **pnpm:** This project uses **pnpm** (<https://pnpm.io/>) for package management.

```
npm install -g pnpm
```

Installation

1. Clone the repository.
2. Install dependencies:

```
pnpm install
```

Environment Setup

1. Copy the example environment file:

```
cp .env.example .env
```

2. Update the `.env` file with your configuration:

- `VITE_API_KEY`: The base URL for your API (e.g., `http://139.59.66.206:5000`)
- `NODE_ENV`: **Set to** `development` for local dev.

Running Locally

Start the development server:

```
pnpm dev
```

The application will be available at `http://localhost:5173` (or the next available port).

□ Scripts

- `pnpm dev`: Starts the development server.
- `pnpm build:dev`: Builds the app for development environment.
- `pnpm build:prod`: Builds the app for production environment.
- `pnpm preview`: Previews the built application.
- `pnpm lint`: Runs ESLint to check for code quality issues.
- `pnpm format`: Formats code using Prettier.
- `pnpm spell-check`: Checks for spelling errors using CSpell.
- `pnpm check:all`: Runs linting, formatting, and spell-checking.

□ Tech Stack

- **Core**: [React 19](https://react.dev/), [Vite](https://vitejs.dev/), [TypeScript](https://www.typescriptlang.org/)
- **State Management**: [Redux Toolkit](https://redux-toolkit.js.org/) & RTK Query
- **UI Framework**: [Material UI \(MUI\) v7](https://mui.com/)
- **Forms & Validation**: [React Hook Form](https://react-hook-form.com/) + [Zod](https://zod.dev/)
- **Routing**: [React Router DOM v7](https://reactrouter.com/)
- **Charts**: [Recharts](https://recharts.org/) & MUI X Charts
- **Localization**: [i18next](https://www.i18next.com/)
- **HTTP Client**: [Axios](https://axios-http.com/)

□ Project Structure

The source code is located in the `src` directory, following a feature-based architecture.

```

src/
├─ app/
│   ├─ common/           # Shared resources
│   │   ├─ @types/       # Global types
│   │   ├─ custom-hooks/ # Reusable React hooks
│   │   ├─ dtos/         # Data Transfer Objects (API types)
│   │   ├─ entities/     # Domain entities/models
│   │   └─ utils/        # Helper functions
│   │
│   ├─ components/       # Reusable UI components
│   │   ├─ crud-template/ # Generic CRUD UI patterns
│   │   └─ ...
│   │
│   ├─ config/           # App configuration (e.g., i18n)
│   │
│   ├─ core/             # Core infrastructure
│   │   ├─ Layout/       # Main app layout (Sidebar, Header)
│   │   ├─ base-query/   # Axios/RTK Query base configuration
│   │   ├─ redux-store/  # Global Redux store setup
│   │   └─ routing/      # Route definitions (Public/Protected)
│   │
│   ├─ features/         # Business logic slices (Redux + API)
│   │   ├─ auth/         # Authentication logic
│   │   ├─ draw/         # Draw management logic
│   │   └─ ...
│   │
│   └─ pages/            # Page components (Views)
│       ├─ dashboard/    # Dashboard & Reporting pages
│       ├─ draw/         # Draw management pages
│       ├─ login/        # Login page
│       └─ ...
│
└─ assets/              # Static assets & Locales

```

□ Architecture Highlights

State Management (Redux Toolkit)

The application uses Redux Toolkit for global state.

- **API Slices:** Located in `src/app/features/*/api`. We use RTK Query for data fetching and caching.
- **State Slices:** Located in `src/app/features/*/state`. Used for local client state (e.g., auth tokens, UI state).

Feature Modules

Each feature (e.g., `auth`, `draw`) typically consists of:

- **API Definition:** Endpoints and query/mutation hooks.
- **DTOs:** TypeScript interfaces for request/response payloads (`src/app/common/dtos`).
- **Entities:** Domain models (`src/app/common/entities`).

Routing

Routing is handled by React Router v7.

- **Protected Routes:** Defined in `src/app/core/routing/protected`. These require authentication.
- **Public Routes:** Defined in `src/app/core/routing/public`.
- **Layout:** The `GlobalLayout` and `Layout` components wrap pages to provide the sidebar and header.

Forms

Forms are built using `react-hook-form` integrated with `zod` for schema validation.

- See `src/app/pages/*/form` for form definitions.

Route Protection & Roles

The application implements a Role-Based Access Control (RBAC) system to protect routes.

How it Works

1. **Authentication Guard:** The `RequireAuth` component (`src/app/core/routing/RequireAuth.tsx`) wraps protected routes. It checks for the existence of an authentication token. If missing, it redirects users to `/login`.
2. **Role-Based Access:**
 - The `MainRouter` (`src/app/core/routing/MainRouter.tsx`) retrieves the current user's role from the Redux store (`authSlice`).
 - It dynamically selects the list of allowed pages using `getPages(role)` from `src/app/core/routing/pagesItems.tsx`.
3. **Route Definitions:**
 - Routes are defined in `src/app/core/routing/protected/ProtectedRoutes.tsx`.
 - **`adminRoutes`:** Contains routes accessible by administrators (e.g., `Dashboard`, `Users`, `Draws`).
 - **`userRoutes`:** Contains routes accessible by standard users.

To add a new protected route:

1. Create your page component in `src/app/pages/`.
2. Import it in `src/app/core/routing/protected/ProtectedRoutes.tsx`.

3. Add it to the `adminRoutes` or `userRoutes` array.

□ Feature Development Strategy

To add a new feature (e.g., "Rewards"), follow this standardized flow:

1. Define Data Transfer Objects (DTOs)

Create DTOs to define the shape of data sent to and received from the API. Use `zod` for schema validation.

- **Location:** `src/app/common/dtos/<feature-name>/`
- **Files:**
 - `create-<feature>.dto.ts`: Schema for creating items.
 - `update-<feature>.dto.ts`: Schema for updating items (often using `.partial()`).
 - `search-<feature>.dto.ts`: Schema for query parameters (pagination, filters).

2. Define Entities

Define the domain model (Entity) representing the resource.

- **Location:** `src/app/common/entities/<feature-name>/`
- **Files:**
 - `<feature>.entity.ts`: The full resource schema.

3. Create API Slice (RTK Query)

Create an API slice to handle network requests.

- **Location:** `src/app/features/<feature-name>/api/<feature>ApiSlice.ts`
- **Steps:**
 1. Add endpoints to `API_ENDPOINTS` in `src/app/core/redux-store/api/api.constants.ts`.
 2. Add a tag to `API_SLICES_TAGS` in `src/app/core/redux-store/api/tags.constant.ts`.
 3. Inject endpoints into `apiSlice` using `injectEndpoints`.
 4. Export the auto-generated hooks (e.g., `useGetRewardsQuery`).

4. Build the UI

Create the page component and form.

- **Location:** `src/app/pages/<feature-name>/`
- **Files:**
 - `<Feature>.tsx`: The main page component (often using `CrudTemplate`).
 - `form/<Feature>FormItems.tsx`: Form fields configuration.

❏ CRUD Template & Forms

The project uses a powerful `CrudTemplate` component to standardize Create, Read, Update, and Delete operations.

`CrudTemplate (src/app/components/crud-template/CrudTemplate.tsx)`

This is a wrapper component that provides a table view, a side drawer for forms, and confirmation dialogs.

Key Props:

- `data`: Array of data to display (from your API hook).
- `columns`: MUI `DataGrid` column definitions.
- `onCreate`, `onEdit`, `onDelete`: Callback functions to handle actions.
- `CreateItems`, `UpdateItems`, `ViewItems`: Components that render the form fields for each mode.
- `enableCreate`, `enableEdit`, `enableDelete`: Booleans to control available actions.

`CrudForm (src/app/components/crud-template/crud-form/CrudForm.tsx)`

Used internally by `CrudTemplate`, but can be customized. It integrates `react-hook-form` with the UI.

How to use:

1. **Define Form Items:** Create a component (e.g., `UserFormItems.tsx`) that renders your input fields (`TextField`, `Select`, etc.). Use `react-hook-form`'s `Controller` or `register` methods.
2. **Pass to Template:** Pass this component to the `CreateItems` or `UpdateItems` prop of `CrudTemplate`.
3. **Validation:** Ensure your form items use the Zod schema defined in your DTOs for validation.

❖❏ Development Workflow

1. **Linting & Formatting:** The project enforces code style using ESLint and Prettier.
 - Run `pnpm check:all` before committing to ensure code quality.
2. **Naming Conventions:**
 - **Components:** PascalCase (e.g., `DrawForm.tsx`)
 - **Hooks:** camelCase with `use` prefix (e.g., `useAppDirection.tsx`)
 - **Utilities:** kebab-case (e.g., `convert-date.ts`)